软件设计实践

软件设计实践笔记

mayun@pku.edu.cn
taoxie@pku.edu.cn

从C到C++(1) 数据抽象与类型系统

1-C++类型系统

Definition

数据类型：一组性质相同的值的集合以及定义在这个值集合上的一组操作

C++是静态类型语言

- 在编译时可确定类型
- 类型标识

```
char s[10] = "hello"; // s为char[10]类型
int* P = &b; // p为int *类型
int *p_1, p_2; // p_1为int *类型, p_2为int类型
```

- 类型别名：`using 别名 = 类型标识`

```
using IntPointer = int*;
IntPointer p = &b;
```

1.1-变量声明与初始化

1.1.1-变量声明

Question

变量声明时发生了什么事情?

1. 找到一块能够承载变量类型所需长度的内存空间
2. 将变量名绑定到这块内存空间
3. 对这块内存空间的内容进行初始化

> **Important**
> 操作未初始化变量可能造成不确定性行为

1.1.2-初始化方式

```
// T为数据类型，v为值
T a1 {v};
T a2 = {v};
T a3 (v);
T a4 = v;
```

C++11的统一初始化： 所有数据类型均可用列表形式初始化
统一初始化的优点：防止窄缩

```
char c1 = 11111; // 警告
char c2{11111};  // 错误
char c3{100};    // 允许6
```

1.2-引用类型

```
int n = 4;
int& r = n; // r引用了，r的类型是int &
r = 3;
std::cout << r << std::endl; // 3
std::cout << n << std::endl; // 3
n = 5;
std::cout << r << std::endl; // 5
```

> **Caution**
>
> 1. 定义引用时一定要初始化为某个变量的引用
> 2. 初始化后，它就一直引用该变量，不能再引用其他变量

```
double a = 4, b = 5;
double& r1 = a;
double& r2 = r1;
r2 = 10;
cout << a << endl; // 10
r1 = b;
cout << a << endl; // 5
```

引用和指针的区别：几乎一样，但引用更方便

```
int n = 4;
int& r = n;
int* p = &n;
```

1.2.1-函数传参的值传递与引用传递

不同的交换方式

```
void falseSwap(int a, int b) {
        int tmp;
        tmp = a;
        a = b;
        b = tmp;
} // 失败
void pointerSwap(int* a, int* b) {
        int tmp;
        tmp = *a;
        *a = *b;
        *b = *tmp;
} // 成功
void referenceSwap(int& a, int& b) {
        int tmp;
        tmp = a;
        a = b;
        b = tmp;
} // 成功
```

1.2.2-引用作为函数的返回值

```
int n = 14;
int& SetValue() {
        return n;
} // 返回引用的函数可以作为左值被赋值
int main() {
        SetValue() = 40;
        std::cout << n << std::endl; // 40
        return 0;
}
```

1.3-类型推导

- `auto`：用于定义变量，编译器可以自动判断变量的类型

```
auto i = 100;     // int
auto p = new A(); // A *
```

```
auto k = 34343LL; // long long
```

2-常量与只读变量

Definition

常量：在编译期间就能确定的值，且在运行期间这个确定的值不会发生变化，例如：

- 字面量
- 字面量构成的式子

只读变量：用const关键词修饰类型说明符的变量，在运行过程中该变量的值不能改变

- const变量必须初始化
- 与常量的关系
  - 并非所有的只读变量都是常量，只有使用常量作为初始化值初始 化的只读变量才是常量

```
const int a {42};
const int b {a};
const int c {a + b};
// 这些变量均为常量
```

但有的只读变量不是常量

```
int a;
std::cin >> a;
const int b {a};
```

声明常量的方法：

```
constexpr int a {42};
```

const的用法

复杂特殊情况

1. 只读引用

```
int n;
const int& r = n;
r = 200; // error
n = 300; // success
```

`r`为`n`的只读引用，只能读取`n`的值，不能通过`r`修改`n`的值，但是`n`本身可以改变

> ⚠ **Caution**
>
> - T &类型的引用，T类型的变量和常量，const T类型的只读变量，都可以用来初始化const T &类型的引用
> - T类型的常量、const T类型的只读变量、const T & 类型的引用则不能用来初始化T &类型的引用，除非进行强制类型转换

2. const指针
3. 指向只读变量的指针 `const T *`
    - 不可以通过该指针修改其指向的内容
    - 变量不需要只读，但编译器认为这样的指针指向的变量是只读的
    - 不能把`const T *`赋值给`T *`，但反过来可以

```
int n = 3. m = 6;
const int* p = &n;
*p = 5; // 错误，指向只读变量的指针不能修改指针对应的值
n = 4; // 正确，变量本身不是只读的，可以修改
p = &m; // 正确，指向只读变量的指针的指向可以变化
// 即，pointer本身不是只读的，但 *pointer 被视为只读变量
```

```
const int* p1;
int* p2;
p1 = p2; // 正确
p2 = p1; // 错误
p2 = (int *)p1; // 强制类型转换后即可
```

2. 只读的指针变量`T *const`
    - 该指针初始化完成后，就不能更改内容，即不允许指向其他对象

```
int buff_size = 512;
int* const pointer = &buff_size;
// pointer将始终指向buff_size

*pointer = 1024;
// 允许通过这个指针修改
// 但是不能修改pointer指向的变量
// 即，pointer本身是只读变量，但 *pointer 可修改

const double pi = 3.14;
const double* const pt_pi = &pi;
```

Tip

函数参数为只读指针时，可避免函数内部不小心改变函数指针所指向部分的内容

- 只读引用也有类似的效果

```
void myPrintf(const char* p) {
        strcpy(p, "this"); // error!
        std::cout << p; // no problem!!
}
```

3-动态内存分配

声明并定义数组时，要求数组的大小必须是**常量**，而且要求是**正整数**

- 注意要求是常量而非只读变量
- 当有初始化器时，可省略不写大小

```
int n = 5;
int a[n] {}; // 严格的编译器会报错

const int m = n;
int a[m] {}; // 严格的编译器会报错
```

Question

如何根据需要开出特定大小的数组?

- 动态分配变量
  - `T`是任意类型名，`p`是类型为`T *`的指针

```
T* p;
p = new T;
```

```
int* pn;
pn = new int;
*pn = 5;
```

- 动态分配数组
  - `T`是任意类型名，`p`是类型为`T *`的指针，`N`是要分配的个数，也可以是unsigned int型表达式

```
T* p;
p = new T[N];
```

```
int* pn;
int i = 5;
pn = new int[i * 20];
p[0] = 20;
// 编译没问题，运行时会越界（index: 0~99）
p[100] = 30;
```

> ⚠ **Warning**
>
> 用 `new` 动态分配的内存空间，一定要用 `delete` 运算符释放

- 如果没有释放，那么即使程序运行结束，这部分内存也不会被回收
  - 即 内存泄露

```
int* p = new int;
*p = 5;
delete p;
delete p; // 异常，一片空间不能被delete多次
```

```
int* p = new int[20];
p[0] = 1;
delete [] p; // 释放数组指针要加[]
```

> **Note**
>
> - 正确性/健壮性
> - 静态类型可以在编译时发现问题
> - 只读变量能防止错误的数据修改
> - 可扩展性
> - 动态内存分配可以按需申请内存空间，应对存储数据规模大小的变化
> - 可复用性
> - 使用变量可以避免程序中重复字面量
> - 可理解性
> - 静态类型可以使开发人员明确知晓每个变量的类型
> - 类型别名可以方便明确变量的类型
> - 自动类型推断能简化开发，但一定程度上破坏了可理解性

从C到C++(1) 过程抽象与函数

函数

函数的声明与定义

- 函数在调用前必须声明
  - 可以先声明后定义
  - 可以多次声明一个名字（但不能多次定义）
- 函数声明时，形参名可以省略

```
void printNum(int x);
// 或者
void printNum(int);

int main() {
        printNum(42);
        return 0;
}

void printNum(int x) {
        std::cout << x << std::endl;
}
```

函数的类型

- 返回值类型(形参类型)

```
void printNum(int x);  // type: void(int)
int max(int x, int y); // type: int(int, int)
```

- 可以用`using`起别名

```
using Intof2Int = int(int, int);

int add(int a, int b);

int main() {
        Intof2Int *funcPtr = &add;
        std::cout << (*funcPtr)(3, 4) << std::endl;

        return 0;
}
```

函数的规约

- 函数实现者和函数使用者之间的约定，包含：
  - 前置条件：对输入参数的规定
  - 后置条件：对函数在不同输入条件下的返回值和其他额外影响的规定
    - return 什么？
    - 修改了什么全局变量？
- 规约的描述方式
  - 函数签名
    - 函数名，参数类型，返回值类型...
  - 函数描述
    - 功能
    - 对参数有什么限制条件
    - 抛出什么异常
    - 其他影响
- 在给定的规约下
  - 函数实现的正确性：对于所有满足前置条件的输入、函数的返回值和额外影响都满足后置条件的约定
  - 函数实现者：根据规约实现
    - 无需关心函数如何调用，可以假设输入始终有效
  - 函数调用者：根据前置条件调用
    - 无需关心函数如何实现，可以假设函数始终正确

函数设计原则

- 每个函数只做一件事情
- 不要写重复代码
- 函数需要具有一定的泛化性

```
double areaSquare(double r) {
        return r * r;
}
double area_circle(double r) {
        return r * r * M_PI;
}
double area_hexagon(double r) {
        return r * r * (3 * sqrt(3) / 2);
}
```

泛化？

```
double area(double r, double shape_constant) {
        return r * r * shape_constant;
}
```

```
double area_square(double r) {
        return area(r, 1);
}

double area_circle(double r) {
        return area(r, M_PI);
}
double area_hexagon(double r) {
        return area(r, (3 * sqrt(3) / 2);
}
```

函数的高级特性

缺省参数

考虑场景

```
void drawCircle(int radius, int x, int y);
```

要画不同颜色的圆

```
void drawCircle(int radius, int x, int y, int color);
```

以前的语句都要加上`BLACK`，但如果

```
void drawCircle(int radius, int x, int y, int color = BLACK);
```

就可以设置缺省值为`BLACK`

- C++中，定义函数时可以给最右边的连续若干个参数指定缺省值；调用函数时会默认初始化相应参数

函数重载

考虑下面的场景

```
void printInt(int i) {
        std::cout << i << std::endl;
}
void printFloat(float f) {
        std::cout << f << std::endl;
}
void printChar(char c) {
        std::cout << c << std::endl;
}
void printCString(const char *s) {
```

```
        std::cout << s << std::endl;
}
```

函数重载

同一作用域中名字相同但形参列表不同的多个函数

- 形参列表不同包括参数个数不同或（和）参数类型不同
- 编译器会根据调用语句中的实参个数和类型判断应该调用哪个函数
  - 但可能会由于类型自动转换或者默认参数导致有多个函数可以匹配而产生二义性问题，ERROR
- 同名函数只有参数列表不同才能算重载，如果仅仅是返回值类型不同则不构成重载

复杂特殊情况

const的用法

- `T`和`const T`会被认为是相同的类型
- `T *`和`T * const`会被认为是相同的类型
- `T &`和`const T &`会被认为是不同的类型
- `T *`和`const T *`会被认为是不同的类型

函数指针

求和 $\sum_{k=1}^{n} k$，$\sum_{k=1}^{n} k^3$，$\sum_{k=1}^{n}(4k-3)(2k+5)$
合并为

$$\mathcal{S}[n, f] = \sum_{k=1}^{n} f(k)$$

Definition

指向函数的指针变量称为“函数指针”

声明形式

```
int (*ptr)(int, int);
```

- 指针名的括号不能省略
- 声明ptr是一个函数指针，及其参数和返回值类型

使用方法

```
int max(int, int);
int (*ptr)(int, int);
ptr = &max;
(*ptr)(3, 5);
```

利用类型别名简化

```
int max(int, int);
using funcType = int(int, int);
funcType* ptr = &max;
(*ptr)(3, 5);
// 也可以（语法糖）
funcType* ptr;
ptr = max;
ptr(3, 5);
```

函数指针作为形参

```
int summation(int n, int (*term)(int)) { // 高阶函数
        int total = 0, k = 1;
        while(k <= n) {
                total += term(k);
                k++;
        }
        return total;
}
```

Lambda表达式

- 只调用一次的简单函数，可以用lambda表达式
  - 匿名函数，可以理解为函数类型的字面量
- 声明方式
  - [捕获列表](参数表) -> 返回值类型 函数体
  - 函数体中可以访问全局变量，但不能访问局部变量，如果想要访问局部变量，需要用捕获列表（使用捕获列表后无法转换为函数指针）
    - [ ] 不使用任何外部变量
    - [=] 外部变量传值
    - [&] 外部变量传引用
    - [x, &y] x传值，y传引用
    - [=, &x, &y] x，y传引用，其余传值
    - [=, x, y] x，y传值，其余传引用
  - 返回值类型可以不写
- 捕获列表示例

```
int main() {
        int x = 100, y = 200, z = 300;
        cout << [](double a, double b) { return a + b; }(1.2, 2.5) << endl;
        auto ff = [=, &y, &z](int n) {
                cout << x << endl;
                y++;
                z++;
                return n * n;
        }
        cout << ff(15) << endl;
        cout << y << "," << z << endl;
}
```

⚠ Warning

没有捕获列表的lambda表达式才能作为函数指针传入高阶函数

Note

- 正确性/鲁棒性
  - 函数规约使得函数实现者和调用者之间消除歧义，减少错误
  - 函数重载存在二义性问题，但静态检查可消除此问题
- 可扩展性
  - 函数规约使得函数实现者和调用者只要遵循规约就可以独立的变化和替代
  - 缺省参数和函数重载可使得增加函数的参数时不会影响已有的函数调用代码
  .- 可复用性
  - 函数使得相同计算逻辑的程序代码无需多次书写（DRY原则）
  - 函数重载使得函数名称可被复用
  - 利用函数指针实现的高阶函数可使得处理逻辑得到更高级别的复用
- 可理解性
  - 良好的函数命名有助于准确理解其功能
  - 查看规约比审查实现更容易理解程序的功能
  - 函数重载可能会降低一定的可理解性，要谨慎考量是否需要重载
  - Lambda表达式虽可以减少代码书写，但一定程度降低了可理解性